第十三章 I/O 系统

1. Consider the following I/O scenarios on a single-user PC.
a. A mouse used with a graphical user interface
b. A tape drive on a multitasking operating system (assume no device preallocation is available)
c. A disk drive containing user files

For each of these I/O scenarios, would you design the operating system to use buffering, spooling, caching, or a combination? Would you use polled I/O, or interrupt-driven I/O? Give reasons for your choices.

a. ① Small Buffering, no spooling or caching.

Reason: Short bursts don't get lost if the GUI thread is briefly busy.
Not necessary to use spooling or caching for a mouse.

② Interrupt-driven.

Reason: Mouse events are sporadic. Polling wastes CPU time.

b. ① Buffering and spooling, no caching.

Reason: Buffering is good for matching process speed and tape speed.
Spooling lets multiple processes submit tape requests as jobs that are executed one at a time.

② Interrupt-driven.

Reason: Tape ops are slow. Polling wastes CPU time.

c. ① Buffering and caching, no spooling.

Reason: Disks have high access latency. Cache to speed up.
Buffering queues and coalesces I/O, helps throughput.

② Interrupt-driven.

Reason: Disk I/Os are slow. Polling wastes CPU time.

2. What are the various kinds of performance overheads associated with servicing an interrupt?

When an interrupt occurs the currently executing process is interrupts and its state is stored in the appropriate process control block. The interrupt service routine is then dispatch in order to deal with the interrupt. On completion of handling of the interrupt, the state of the process is restored and the process is resumed. Therefore, the performance overheads include the cost of saving and restoring process state and the cost of flushing the instruction pipeline and restoring the instructions into the pipeline when the process is restarted.